

Ošetření chyb

Karel Richta a kol.

Katedra technických studií
Vysoká škola polytechnická v Jihlava

© Richta;Hořeňovský;Hrabalík, 2020

Základy strukturovaného programování, ZSP
11/2020, Lekce 7

<https://moodle.vspj.cz/course/view.php?id=200871>

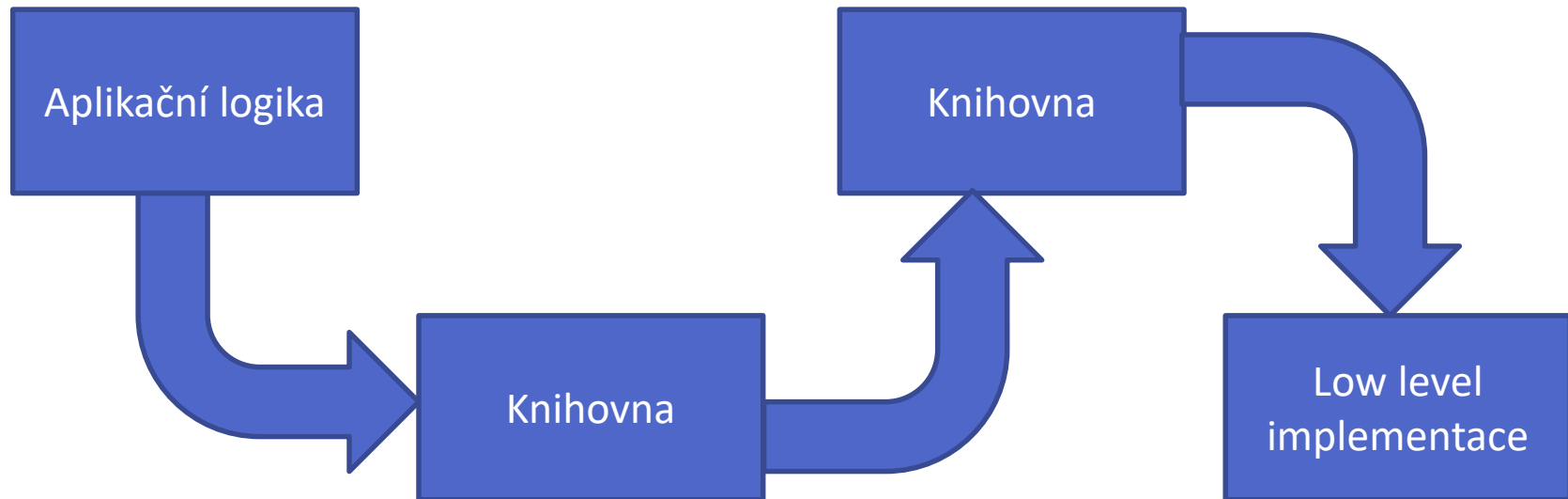


Vysoká škola
polytechnická
Jihlava



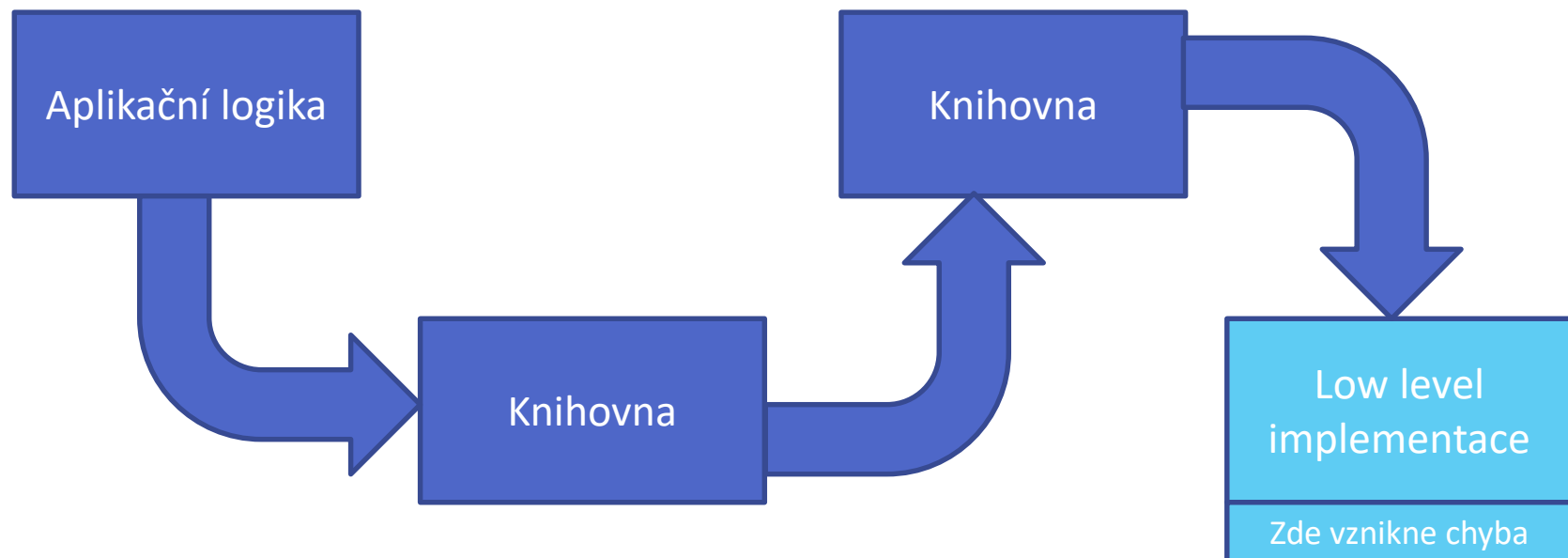
Problém

- Při běhu programu mohou vznikat různé chyby.
 - Například nemusíme mít práva ke čtení/zápisu souboru.
- Tyto chyby je potřeba nějak ošetřit.
- Místo, které si umí s chybou poradit, je obvykle jiné, než místo, kde chyba vznikla.



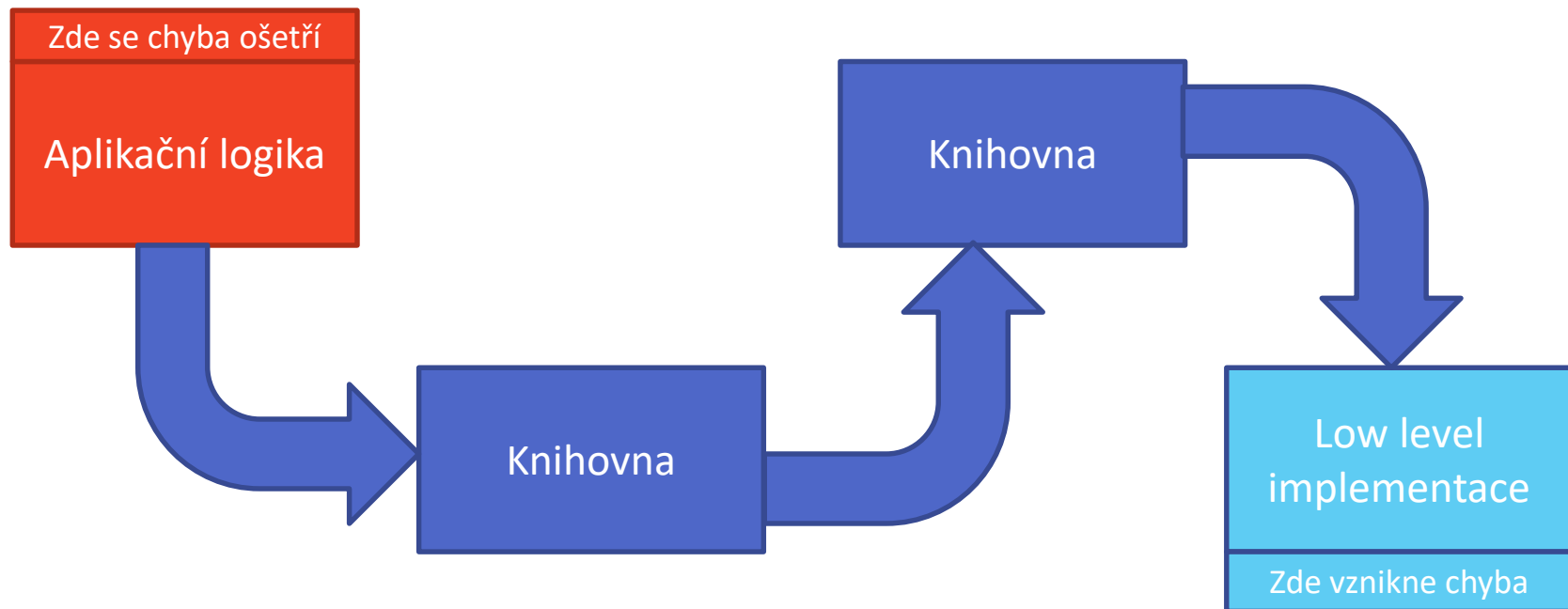
Problém

- Při běhu programu mohou vznikat různé chyby.
 - Například nemusíme mít práva ke čtení/zápisu souboru.
- Tyto chyby je potřeba nějak ošetřit.
- Místo, které si umí s chybou poradit, je obvykle jiné, než místo, kde chyba vznikla.



Problém

- Při běhu programu mohou vznikat různé chyby.
 - Například nemusíme mít práva ke čtení/zápisu souboru.
- Tyto chyby je potřeba nějak ošetřit.
- Místo, které si umí s chybou poradit, je obvykle jiné, než místo, kde chyba vznikla.



Důsledky

- K řešení chyb je tedy potřeba přenést informace o chybě z nižších vrstev do vyšších.
- Existují 3 různé způsoby:
 - Návratové hodnoty
 - Chybové příznaky
 - Výjimky
- Ve standardní knihovně C++ se vyskytují všechny tři.
 - Části převzaté z C obvykle používají návratové hodnoty.
 - Matematické funkce používají chybové příznaky.
 - STL používá výjimky.

Řešení pomocí chybových příznaků

- Základní schema:

```
int chyba = 0; // globální chybový příznak

void f(parametry) {
    // část kódu, kde může nastat chyba
    // pokud nastane:
    // - nastaví se chybový příznak na dohodnutou nenulovou hodnotu
    // - přeruší se normální zpracování a funkce se ukončí
    // pokud chyba nenastane, nic se neděje, příznak se nenastaví
};

int main() {
    f(skutečné parametry);
    if (chyba) { zpracování chyby }
    // normální zpracování
}
```

Chybové příznaky

- Funkce, která narazí na chybu, ji zapíše do globální proměnné.
- Funkce, která se chce o chybě dozvědět, zkontroluje zapsanou hodnotu.
- Chybové příznaky se používají v matematické knihovně a v POSIX standardu.
 - Např. pokud funkce `log` narazí na chybu, třeba negativní vstup, zapíše do `errno` hodnotu `EDOM`.
 - Funkce volající `log` má následně možnost `errno` přečíst.
- Nemění signaturu funkce.

```
int main() {  
    double foo = std::log(-1.23);  
    if (errno == EDOM) {  
        printf("log() has failed because: %s\n", strerror(errno));  
    }  
}
```

```
>>> log() has failed because: Numerical argument out of domain
```

Chybové příznaky – knihovna errno.h

- Definuje makro `errno` – lze si představit jako deklaraci proměnné `int errno = 0;`
- Obsahuje kód poslední zjištěné chyby (0 znamená OK).
- Nejméně tři konstanty - kódy: EDOM (domain error), ERANGE (range error) a EILSEQ (illegal sequence).
- Pokud některá funkce narazí na chybu, např. log na negativní vstup, zapíše do `errno` kód chyby, zde např. hodnotu EDOM.
- Funkce volající log má následně možnost `errno` přečíst.

```
enum class errc;
```

```
http://en.cppreference.com/w/cpp/error/errc
```


Chybové příznaky – Problémy

- Chyby mohou být ignorovány.
 - Programátor musí vynaložit úsilí, aby ignorovány nebyly.
- **Není jisté, kdy a kde k chybě došlo.**
 - Pokud po volání funkce nekontrolujeme `errno`, už se nedá určit, zda k chybě došlo.
 - Ani není možné zjistit, jestli se chyb nevyskytlo více.
- **Je to spousta kódu, která se špatně čte a špatně píše.**
 - Volání funkce následuje mnoho řádků, které čtou `errno` a reagují na chybu.
 - Funkce se dají skládat, ale pouze za cenu možného přehlédnutí chyby.

```
double sumlogs(const double numbers[],int i1,int i2) {  
    double result = 0;  
    for (auto n = i1; n <= i2; n++) result += log(numbers[n]);  
    return result;  
}
```

Chybové příznaky – Problémy

- Chyby mohou být ignorovány.
 - Programátor musí vynaložit úsilí, aby ignorovány nebyly.

```
double sumlogs(const double numbers[],int i1,int i2) {  
    double result = 0;  
    for (auto n = i1; n <= i2; n++) result += log(numbers[n]);  
    return result;  
}  
  
double foo(const double numbers[], int i1, int i2) {  
    return exp(sumlogs(numbers,i1,i2));  
}
```

- Když zavoláme log s neplatným parametrem, nastaví se errno. To samé platí o exp.
- Kde došlo k chybě?
 - Nevíme.

Chybové příznaky – Problémy

- **Není jisté, kdy a kde k chybě došlo.**
 - Pokud po volání funkce nekontrolujeme `errno`, už se nedá určit, zda k chybě došlo.

```
// Toto už zkontroluje chybu, ale neví kde nastala
double sumlogs(const double numbers[],int i1,int i2) {
    double result = 0;
    for (auto n = i1; n <= i2; n++) {
        result += log(numbers[n]);
    }
    if (errno == EDOM) {
        // handle error
    }
    return result;
}
```

- Chybu kontrolujeme, ale nevíme, kdy k ní došlo.
 - V tomto případě to nevadí, ale jindy může.

Chybové příznaky – Problémy

- Je to spousta kódu, která se špatně čte a špatně píše.
 - Volání funkce následuje mnoho řádků, které čtou errno a reagují na chybu.
 - Funkce se dají skládat, ale pouze za cenu možného přehlédnutí chyby.

```
double sumlogs(const double numbers[],int i1,int i2) {
    double result = 0;
    for (auto n = i1; n <= i2; n++) {
        result += log(numbers[n]);
    }
    if (errno == EDOM) {
        fprintf(stderr, "Doslo k chybe v sumlogs: EDOM\n");
        return NAN;
    }
    return result;
}

double foo(const double numbers[], int i1, int i2) {
    auto temp = sumlogs(numbers, i1, i2);
    if (errno == EDOM || isnan(temp)) {
        fprintf(stderr, "foo chyba: suma logaritmu skončila chybou\n");
        return NAN;
    }
    auto res = std::exp(temp);
    if (errno == ERANGE) {
        fprintf(stderr, "foo chyba: vysledek je prilis velky.\n");
    }
    return res;
}
```

Zvětšeno na příštím
snímku

Chybové příznaky – Problémy

```
double sumlogs(const double numbers[],int i1,int i2) {
    double result = 0;
    for (auto n = i1; n <= i2; n++) {
        result += log(numbers[n]);
    }
    if (errno == EDOM) {
        fprintf(stderr, "Doslo k chybe v sumlogs: EDOM\n");
        return NAN;
    }
    return result;
}

double foo(const double numbers[], int i1, int i2) {
    auto temp = sumlogs(numbers, i1, i2);
    if (errno == EDOM || isnan(temp)) {
        fprintf(stderr, "foo chyba: suma logaritmu skončila chybou\n");
        return NAN;
    }
    auto res = std::exp(temp);
    if (errno == ERANGE) {
        fprintf(stderr, "foo chyba: vysledek je prilis velky.\n");
        return NAN;
    }
    return res;
}
```

Chybové příznaky – Problémy

```
double sumlogs(const double numbers[],int i1,int i2) {
    double result = 0;
    for (auto n = i1; n <= i2; n++) {
        result += log(numbers[n]);
    }
    if (errno == EDOM) {
        fprintf(stderr, "Doslo k chybe v sumlogs: EDOM\n");
        return NAN;
    }
    return result;
}

double foo(const double numbers[], int i1, int i2) {
    auto temp = sumlogs(numbers, i1, i2);
    if (errno == EDOM || isnan(temp)) {
        fprintf(stderr, "foo chyba: suma logaritmu skončila chybou\n");
        return NAN;
    }
    auto res = std::exp(temp);
    if (errno == ERANGE) {
        fprintf(stderr, "foo chyba: vysledek je prilis velky.\n");
        return NAN;
    }
    return res;
}
```

Řešení pomocí návratových hodnot

- Základní schema:

```
int f(parametry) {  
    // část kódu, kde může nastat chyba  
    // pokud nastane:  
    // - přeruší se normální zpracování a funkce se ukončí  
    // - návratová hodnota indikuje chybu  
    // pokud chyba nenastane, funkce vrátí hodnotu 0  
};  
  
int main() {  
    if (f(skutečné parametry)) { zpracování chyby }  
    // normální zpracování  
}
```

Řešení pomocí návratových hodnot

- Funkce, které vrací numerickou hodnotu (tradičně `int`)
 - Návratová hodnota je pak určuje chybu, ke které došlo při průběhu funkce.
 - Hodnoty značící chybu (nebo úspěch) jsou předem definovány.
- Funkce často přijímají i tzv. výstupní parametr.
- Navracené hodnoty z funkce se pak vrací výš.

```
int main() {  
    printf("Kolik cisel bude nasledovat?\n");  
    int n;  
    int res = scanf("%d", &n);  
    if (res != 1) {  
        printf("To neni cislo\n");  
        exit(1);  
    }  
    ...  
}
```


Návratové hodnoty – Problémy

- Chyby mohou být ignorovány.
 - Programátor musí vynaložit úsilí, aby ignorovány nebyly.
- Pokud někdy ignorujete chybu, její hlášení je navždy ztraceno.
- Je to spousta kódu, která se špatně čte a špatně píše.
 - Vracená hodnota z každé funkce se musí uložit, zkontrolovat, vyřešit a případně vrátit z funkce.
 - Funkce se nedají skládat.

```
int main() {  
    printf("Kolik cisel bude nasledovat?\n");  
    int n;  
    scanf("%d", &n);  
    for (int i = 0; i < n; ++i) {  
        ...  
    }  
}
```

Návratové hodnoty – Problémy

- Chyby mohou být ignorovány.
 - Pokud někdy ignorujete chybu, její hlášení je navždy ztraceno.

```
int nacti_cislo() {  
    int temp;  
    scanf("%d", &temp);  
    return temp;  
}  
  
int main() {  
    printf("Kolik cisel bude nasledovat?\n");  
    int n = nacti_cislo();  
    ...  
}
```

- Pokud se nepovedlo načíst číslo, v n je podivná hodnota.
 - Zbytek programu o tom neví.

Návratové hodnoty – Problémy

- Chyby mohou být ignorovány.
 - Programátor vynaložit úsilí, aby ignorovány nebyly.

```
int nacti_kladne_cislo() {  
    int temp;  
    int res = scanf("%d", &temp);  
    if (res != 1) {  
        return -1;  
    }  
    if (temp < 0) {  
        return -2;  
    }  
    return temp;  
}
```

- Už nejsme schopni vrátit všechny možné hodnoty datového typu `int`.
 - Musíme vybrat, které hodnoty obětujeme.

Návratové hodnoty – Problémy

- Je to spousta kódu, který se špatně čte i píše.
 - Vrácená hodnota z každé funkce se musí uložit a zkontrolovat; v případě chyby musíme správně zareagovat.
 - Funkce se nedají skládat.

```
int nacti_kladne_cislo() {
    int temp;
    int res = scanf("%d", &temp);
    if (res != 1) {
        return -1;
    }
    if (temp < 0) {
        return -2;
    }
    return temp;
}

int main() {
    printf("Kolik cisel bude nasledovat?\n");
    int cislo = nacti_kladne_cislo();
    if (cislo < 0) {
        if (cislo == -2) {
            printf("Potrebuju KLADNE cislo.\n");
        }
        if (cislo == -1) {
            printf("Potrebuju cislo.\n");
        }
        exit(1);
    }
    ...
}
```

Zvětšeno na příštím
snímku

```
int nacti_kladne_cislo() {
    int temp;
    int res = scanf("%d", &temp);
    if (res != 1) {
        return -1;
    }
    if (temp < 0) {
        return -2;
    }
    return temp;
}

int main() {
    printf("Kolik cisel bude nasledovat?\n");
    int cislo = nacti_kladne_cislo();
    if (cislo < 0) {
        if (cislo == -2) {
            printf("Potrebuju KLADNE cislo.\n");
        }
        if (cislo == -1) {
            printf("Potrebuju cislo.\n");
        }
        exit(1);
    }
    ...
}
```

```

int nacti_kladne_cislo() {
    int temp;
    int res = scanf("%d", &temp);
    if (res != 1) {
        return -1;
    }
    if (temp < 0) {
        return -2;
    }
    return temp;
}

int main() {
    printf("Kolik cisel bude nasledovat?\n");
    int cislo = nacti_kladne_cislo();
    if (cislo < 0) {
        if (cislo == -2) {
            printf("Potrebuju KLADNE cislo.\n");
        }
        if (cislo == -1) {
            printf("Potrebuju cislo.\n");
        }
        exit(1);
    }
    ...
}

```

Řešení pomocí výjimek (C++)

- Základní schema:

```
int main() {  
    try {  
        // část kódu, kde může nastat chyba  
        // pokud nastane - způsobí změnu ve zpracování, tzv. „hodí výjimku“  
        // - přeruší se normální zpracování, hledá se ovladač této výjimky  
        // - výjimka je identifikována pomocí typu  
        // pokud chyba nenastane, nic se neděje  
    } catch (... typ výjimky...) {  
        // zpracování výjimky daného typu  
    } catch (... jiný typ výjimky...) {  
        // zpracování výjimky jiného typu  
    }  
    // další případné ovladače  
}
```

Výjimky

- Chyba nemůže být ignorována.
 - Ignorování chyby ukončí proces.
- Funkce, která hlásí chybu, hodí výjimku.
- Funkce, která umí chybu zpracovat, ji chytá.
- Funkce se dají skládat.

```
int main() {  
    try {  
        while (true) {  
            new int[100000000u1];  
        }  
    } catch (const std::bad_alloc& e) {  
        std::cout << "Allocation failed: " << e.what() << '\n';  
    }  
}
```


The End